C Programming

Characters & Strings: Ch.12

char arrays | the NULL terminator | string.h functions

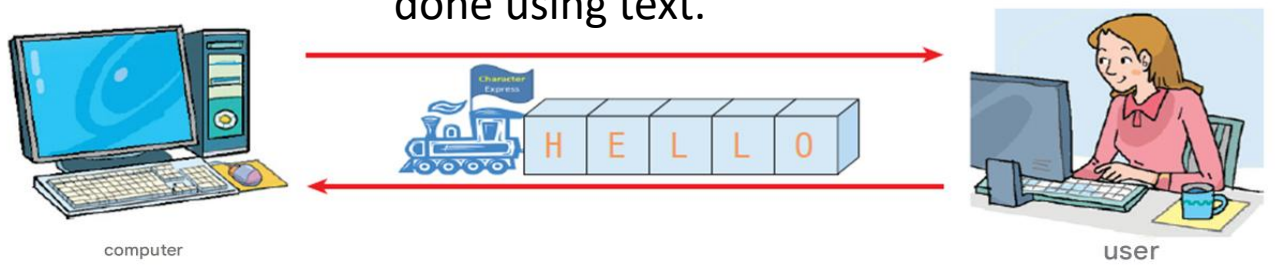
What You Will Learn in This Chapter

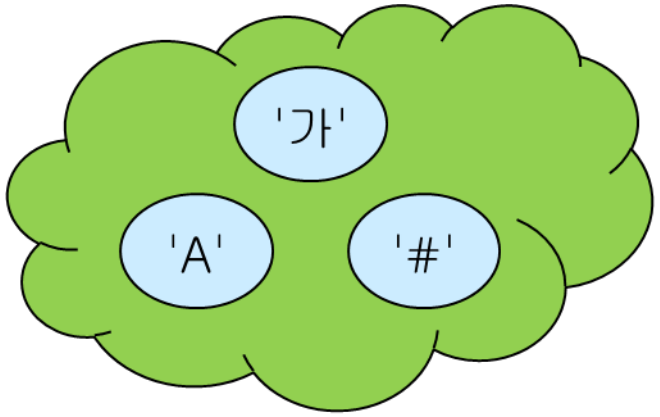
- How to express characters, how to represent strings
- What is a **string**? Input/output of strings
- Character processing library functions
- **String** processing library functions
- Since humans use characters to express information, strings occupy an important position in programs.

How to Represent Strings

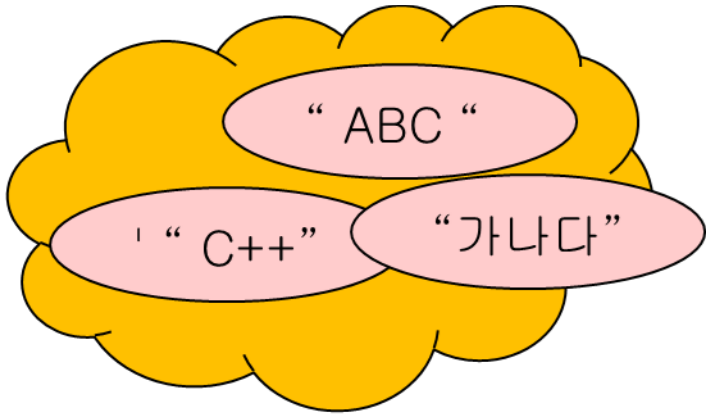
- **String**: a collection of several characters, such as "A", "Hello World!"
- One character is stored in a char type variable.
- Strings are stored as char type arrays: H E L L O

A lot of computer work is done using text.





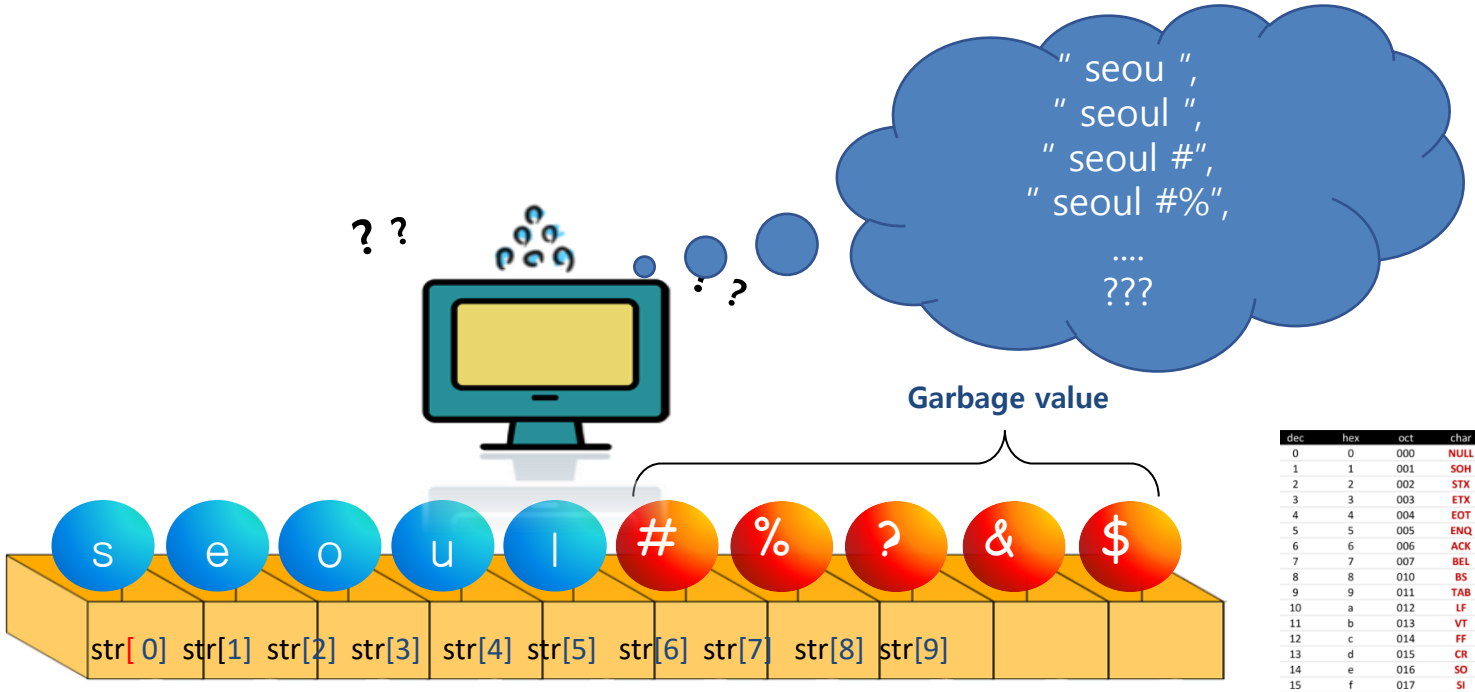
Character



String

How to Represent Strings: Why do we need to mark the end of a string?

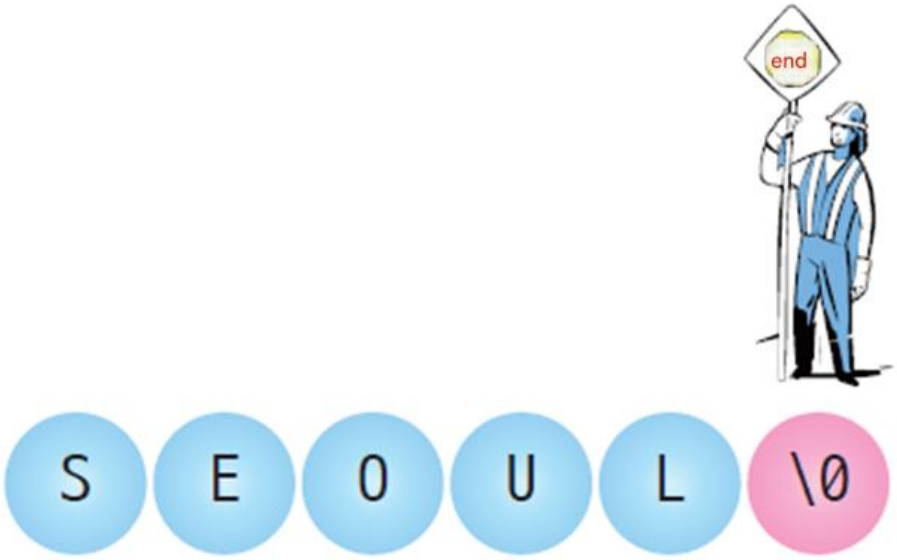
- Since we don't know where the string ends, we need to mark it.
  - A garbage value may represent any character.

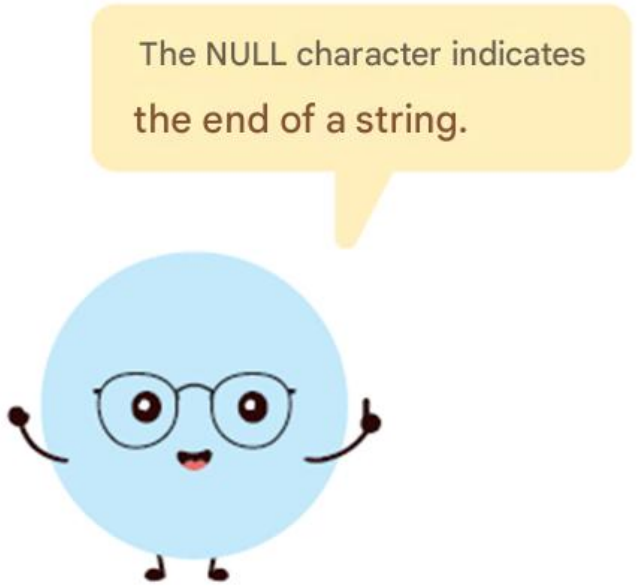


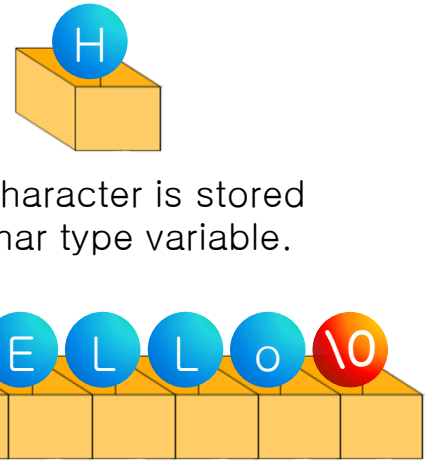
dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

How to Represent Strings: NULL Character

- NULL character: indicates the end of a **string**. (They are related but different concepts.)
  '\0' == null character
  NULL == null pointer
  0 == value or null pointer
- Why do we need to mark the end of a **string**?
  Since we don't know where the **string** ends otherwise, we need to mark it.







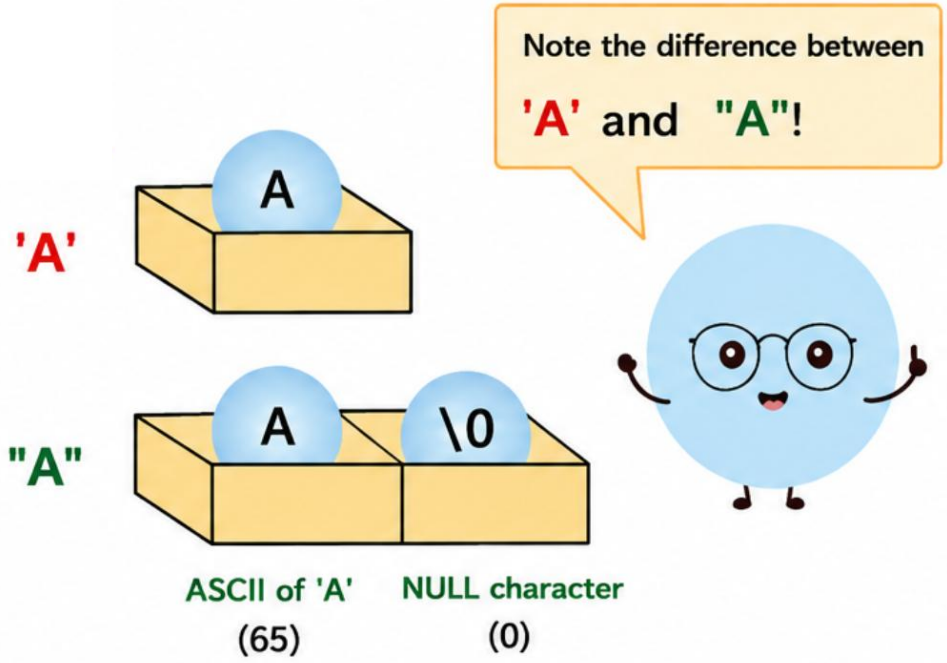
Strings are stored as char type arrays

How to Represent Strings: Difference A, ‘A’, “A”

A: The compiler considers A as the name of a variable.

'A': Represents the letter A.

"A": Represents a string consisting only of the letter A. This is different from 'A'.



- The thing to note here is the difference between 'A' and "A".
- 'A' represents a single character and is the same as the ASCII code for the character A.
- "A" is a string, and the NULL character is added to the ASCII code of A to indicate the end of the string.

'A' (single character) : Stored as a single byte → ASCII value of 'A' (65)

"A" (string) : Stored as two bytes → 'A' (65) followed by NULL character '\0' (0)

Example: Printing Characters One by One

```c
#include < stdio.h >

int main( void )
{
    int i ;
    char str [4];
    str [0] = 'a';
    str [1] = 'b';
    str [2] = 'c';
    str [3] = '\0';

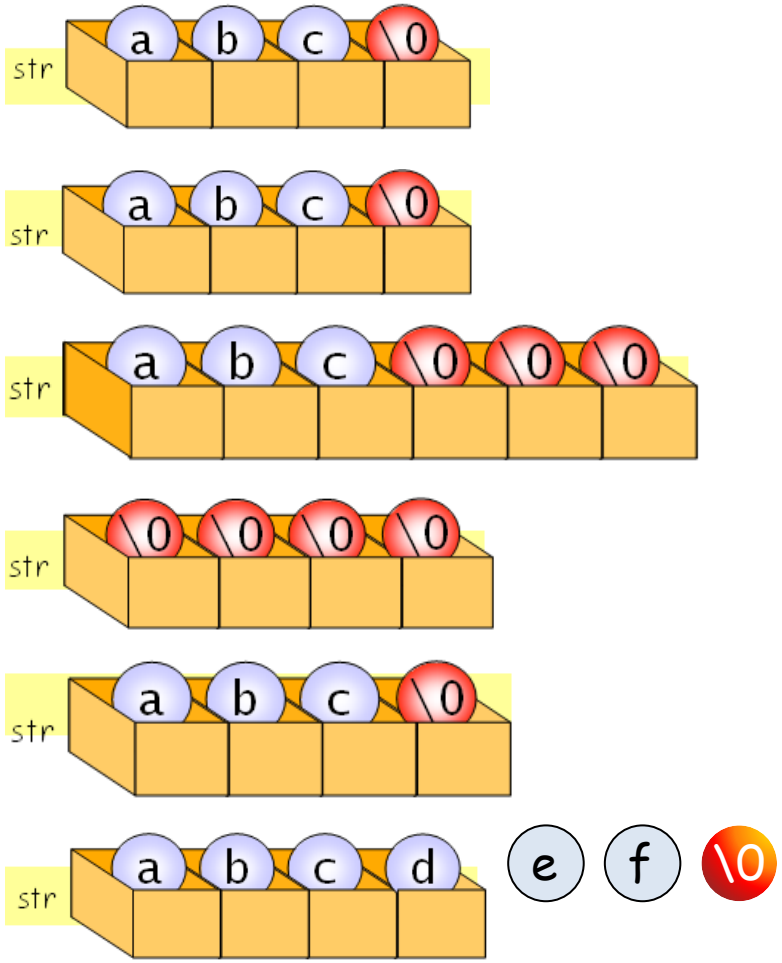
    i = 0;
    while ( str [ i ] != '\0' ) {
        printf ( "%c" , str [ i ]);
        i ++;
    }
    return 0;
}
```

```
abc
```

We print out characters one by one, stopping the loop when a NULL character is encountered.

Initializing a character array (String)

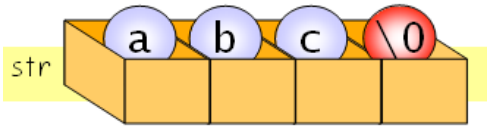
- char str [4] = { 'a', 'b', 'c', '\0' };
- char str [4] = "abc";
- char str [6] = "abc";
- char str [4] = "";
- char str [] = "abc";
- char str [4] = "abcdef"; // Error
- char str []; // Error: The compiler doesn't know how much memory to allocate



Output of Strings

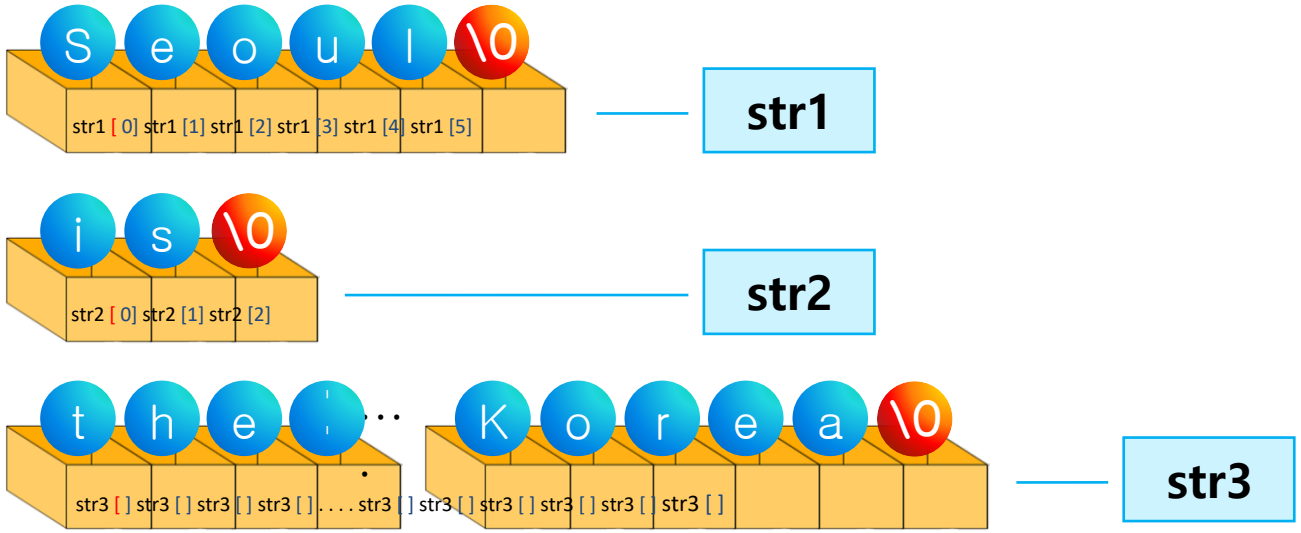
```
char str [] = “abc”;
printf (“%s”, str );
```






```
#include < stdio.h >

int main( void )
{
    char str1[6] = "Seoul" ;
    char str2[3] = { ' i ' , 's' , '\0' };
    char str3[] = "the capital city of Korea." ;
    printf ( "%s %s %s\n" , str1, str2, str3);
    return 0;
}
```





String Copy

VSC

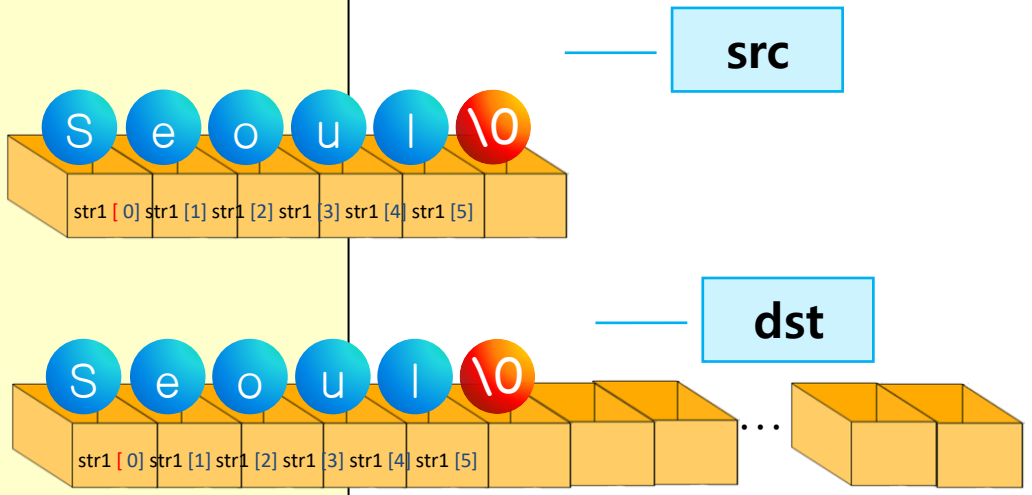
```
#include < stdio.h >

int main( void )
{
        char src [] = “Seoul" ;
        char dst [100];

        int i ;
        printf ( " Original string =%s\n" , src );
        for (i = 0; src[i] != '\0' ; i++)
            dst [ i ] = src [ i ];

        dst [ i ] = '\0' ;
        printf ( " Copied string =%s\n" , dst );

        return 0;
}
```




```
Original string = Seoul
Copied string = Seoul
```

Counting String Length

VSC

```
// Program to find the length of a string
#include < stdio.h >

int main( void )
{
        char str [30] = "C language is easy";
        int i = 0;

        while ( str [ i ] != 0) {
             i ++;
        }
        printf ( "The length of the string \"%s\" is %d.\n" , str , i );
        return 0;
}
```



Changing a Character Array

In C, a character array can be modified after initialization, but a new string cannot be assigned directly to an array using =.

- Correct Way

Method 1: Modify individual characters

```
char str[10] = "Hello";

str[0] = 'W';
str[1] = 'o';
str[2] = 'r';
str[3] = 'l';
str[4] = 'd';
```

Method 2: Use strcpy() (Recommended)

```
#include <string.h>

char str[10] = "Hello";
strcpy(str, "World");  // Correct
```

- Incorrect Way

An array name is not a modifiable lvalue. Therefore, a new string cannot be assigned directly to an array.

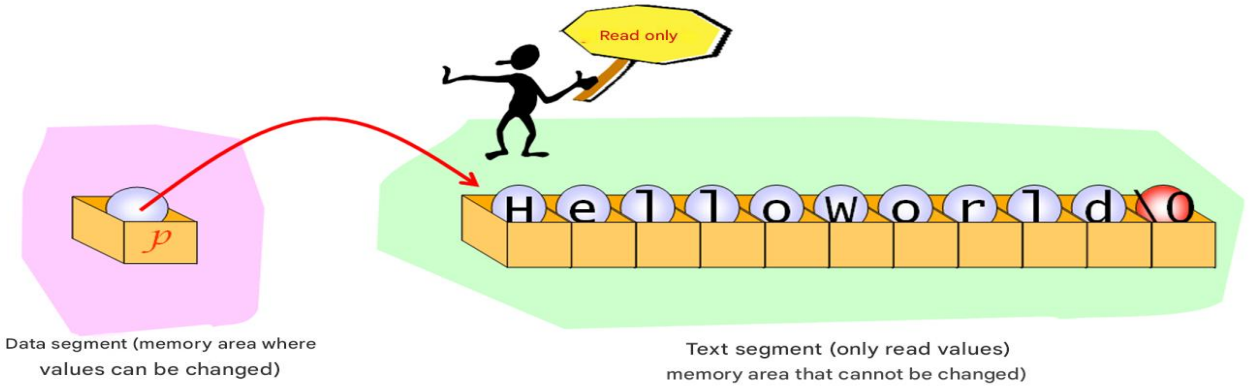
```
char str[10] = "Hello";
str = "World";  // Error!
```

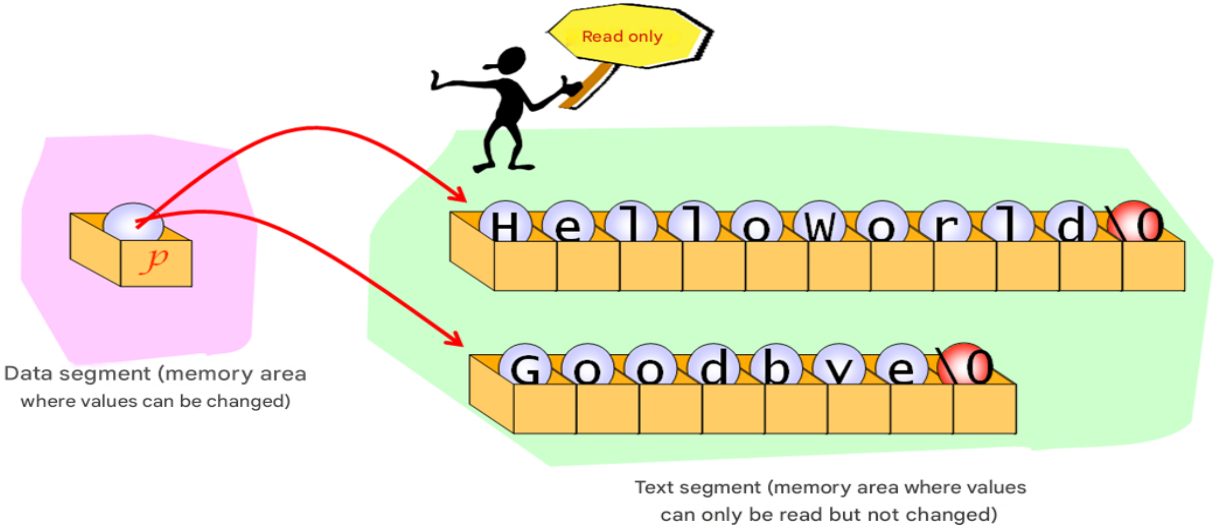
* An array name can be used to assign the array's starting address to a pointer variable, but a new address cannot be assigned to the array name.

```
char *str = "Hello";
str = "World";       // Correct
```

String Constant

- **String** constant: a **string** included in the program source code, such as "HelloWorld".
  - **String** constants are stored in the text segment (Read-Only area).
- char *p = "HelloWorld";
  - p points to a **string** constant.
- p = "Goodbye"; (reassigning p itself) is OK
  - p can point to a different **string**.





Character Array vs. Character Pointer

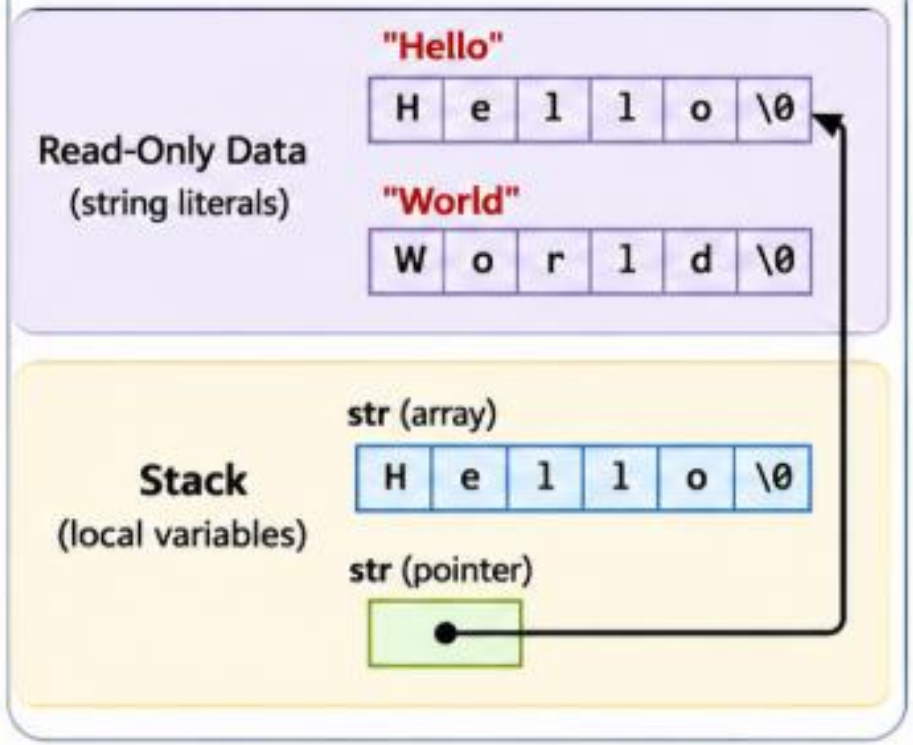
Character Array

```
char str[] = "Hello";
```

- Stores the characters directly in its own memory.
- Characters can be modified (read/write).
- Cannot be assigned a new address.

```
✔ str[0] = 'W';    // OK
✘ str = "World";   // Error!
```

Memory (simplified)



Character Pointer

```
char *str = "Hello";
```

- Stores the address of the first character (a pointer).
- Can be reassigned to point to another string.
- The string literal is typically read-only (should not be modified).

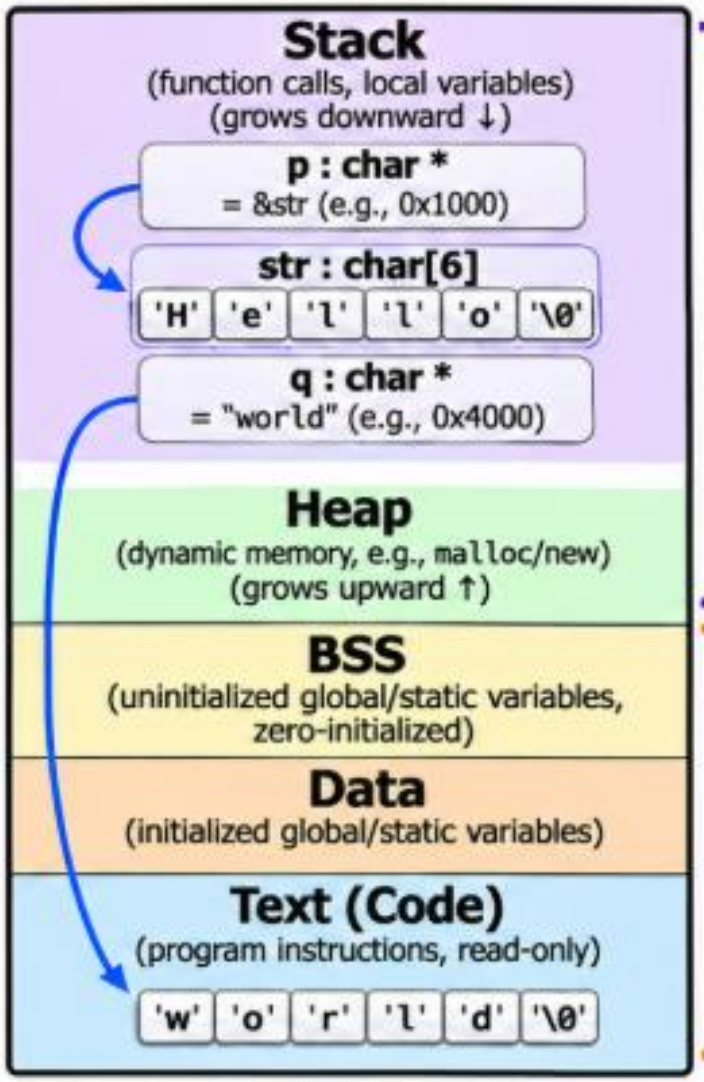
```
✔ str = "World";   // OK
✘ str[0] = 'W';    // Undefined behavior!
```

Key Takeaways

- An array stores characters (the data itself).
- A pointer stores an address (points to character data).
- A pointer can be reassigned to another address, but an array name cannot.

Character Array vs. Character Pointer

A Program's Data Is Stored in Different Memory Regions Based on Its Purpose!



Size determined at **runtime** (execution time)

Size determined at **compile/link time**

Example (Local Variables)

```
void func() {
    char str[] = "Hello";
    char *p = str;
    char *q = "world";
}
```

Where They Are Stored

- `str` : stored in the **Stack** as a local array, `{'H','e','l','l','o','\0'}` (6 bytes).
- `p` : stored in the **Stack** as a pointer to `str[0]` (e.g., `p = &str`, 0x1000).
- `q` (`char *`): stored in the **Stack** as a pointer to the string literal `"world"` (e.g., 0x4000).
- `"world"` : stored in the **Text** (read-only) section as `{'w','o','r','l','d','\0'}` (6 bytes).

Key Takeaways

- **Text (Code), Data**, and **BSS**: sizes are determined at **compile/link time** (from the program).
- **Stack** and **Heap**: sizes are determined at **runtime** (during execution) and can grow or shrink.

Character Array vs. Character Pointer

The compiler uses a symbol table to map the name ‘str’ to its address.

1 Source Code (C)

```
char str[] = "Hello";
char *p = str;
```

2 Compilation (simplified)

Lexical / Syntax Analysis

↓

Build Symbol Table

↓

Type Checking

↓

Generate Machine Code

3 Symbol Table (during compilation)

Name	Type	Size	Address*
str	char[6]	6	0x7FF4
p	char *	8	0x7FFC

* Example addresses (conceptual)

4 Generated Code (conceptual)

`str[0] = 'W';` → Use the address of str from the symbol table (e.g., 0x7FF4).

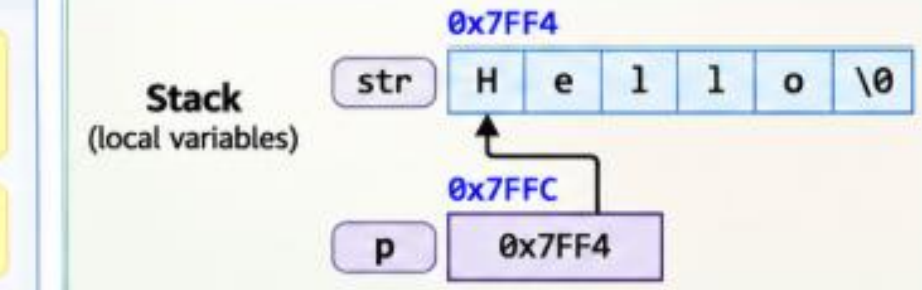
`p = str;` → Store the address of str (0x7FF4) into p.

5 Memory Layout (runtime)

Character Array: `char str[] = "Hello";`



Character Pointer: `char *p = str;`



Key Points

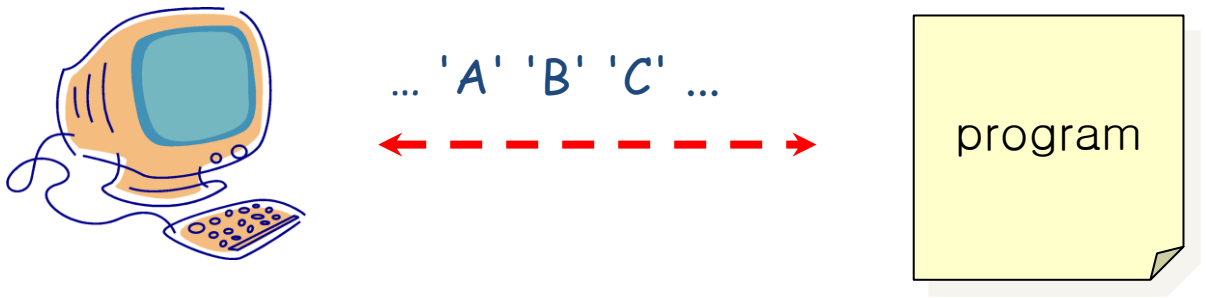
- The compiler creates a symbol table that records each name (e.g., `str`), its type (e.g., `char[6]`), size, and the memory address (assigned during code generation).
- When you use 'str' in the code (e.g., `str[0]` or `p = str;`), the compiler looks up 'str' in the symbol table and uses its address to generate the appropriate memory access code.

How to read & write characters

Character input/output library

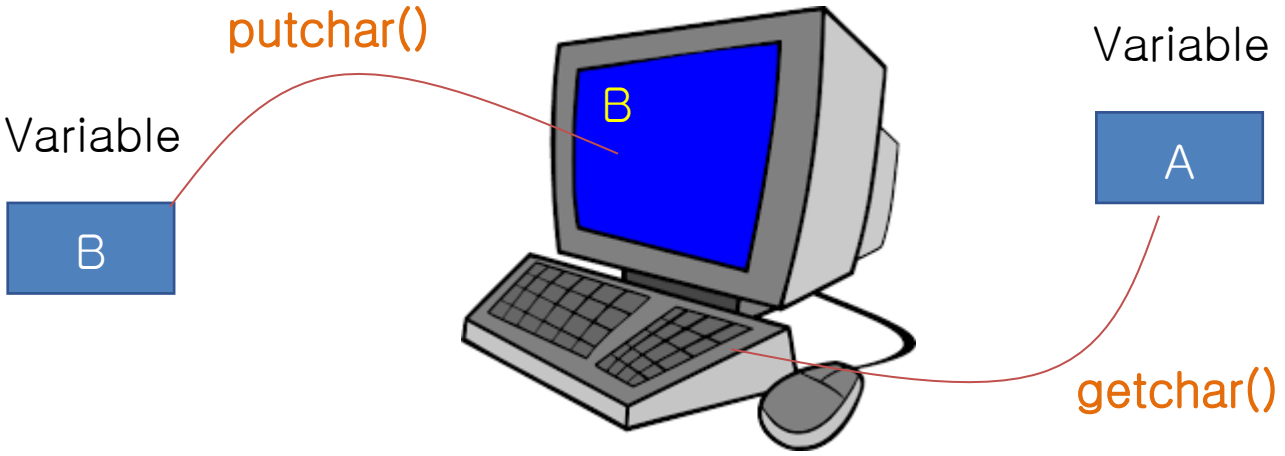
Character Input/Output Library

Input/output functions	explanation
int getchar(void)	Reads and returns a single character.
void putchar(int c)	Prints the character stored in variable c.
int getch(void)	Reads and returns a single character (without using a buffer).
void putch(int c)	Prints the character stored in variable c (without using a buffer).



Character Input/Output: getchar(), putchar()

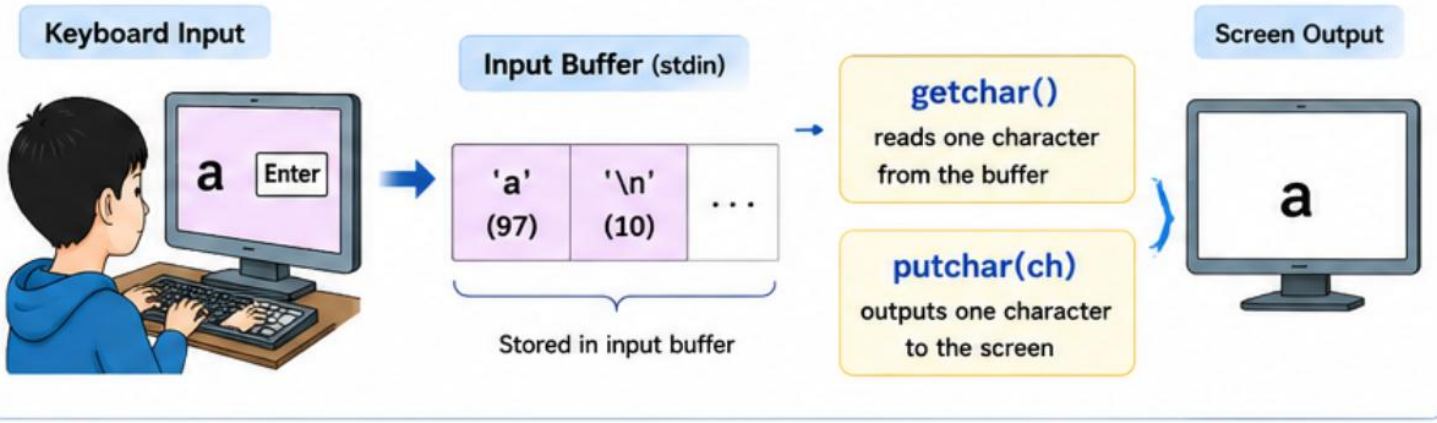
- EOF (End Of File): a special symbol indicating the end of input. EOF is the integer -1.
- getchar() returns EOF if the end of input is reached, or an input error occurs.
- The return type of getchar() is "int" (not char), to distinguish a real ASCII code from EOF.



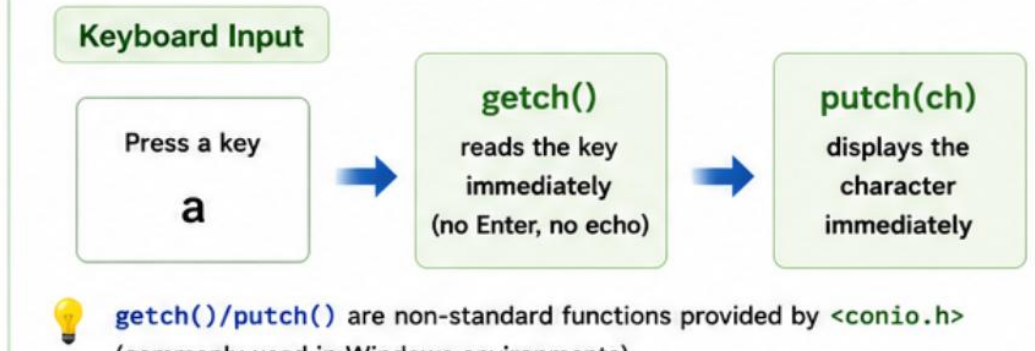
Character Input/Output: getchar(), putchar()

Function	Prototype	Description	Header / Notes
getchar()	`int getchar(void);`	• Reads one character from the standard input (keyboard). • Returns the character as an `int`, • Enter key generates `'\n'` (newline).	`#include <stdio.h>` Standard C function.
putchar(ch)	`int putchar(int ch);`	• Writes one character (`ch`) to the standard output (screen). • Returns the character written as an `int`, or EOF on error.	`#include <stdio.h>` Standard C function.
getch()	`int getch(void);`	• Reads one character from the keyboard without waiting for Enter. • The character is not echoed (not displayed).	`#include <conio.h>` Non-standard (Windows).
putch(ch)	`int putch(int ch);`	• Writes one character to the screen without buffering. • Returns the character written, or EOF on error.	`#include <conio.h>` Non-standard (Windows).

How getchar() and putchar() work



How getch() and putch() work (Windows)



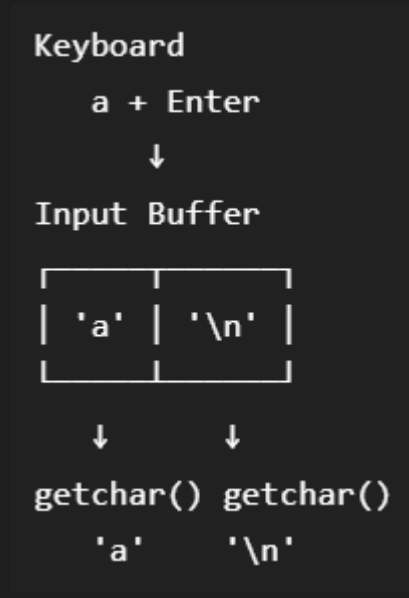
`getch()/putch()` are non-standard functions provided by `<conio.h>` (commonly used in Windows environments).

getchar() / putchar() Example

```c
#include < stdio.h >
int main( void )
{
    int ch ;  // Note the use of int, not char
    while ( ( ch = getchar ()) != EOF )
        putchar ( ch );
    return 0;
}


// To terminate while loop
    while ( ( ch = getchar ()) != EOF && ch != ‘q’)
```

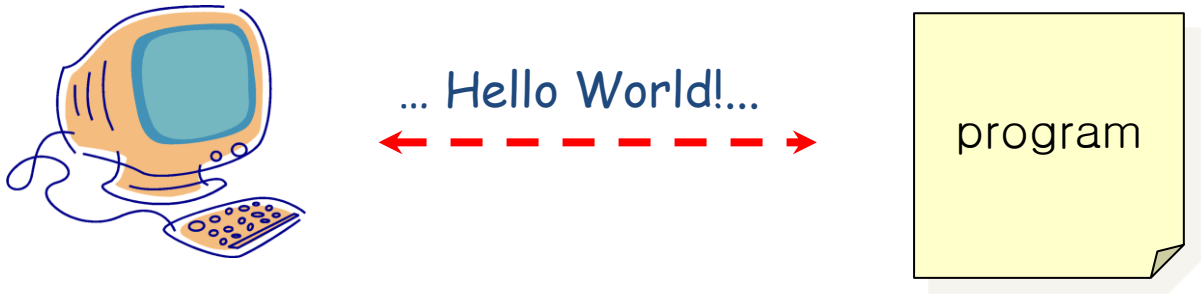
getchar() : returns EOF if the end of input is reached, or an input error occurs




```
a
a
b
b
^Z
```

String Input/Output Library

Input/output functions	explanation
char * gets_s(char *s, int size)	Read a line of string and store it in the character array s[].
int puts(const char *s)	Prints a single line of string stored in array s[].
scanf("%c", &c)	Read one character and store it in variable c.
printf("%c", c);	Prints the character stored in variable c.



String Input/Output Library: gets_s(), puts()

```
char buf[100];

gets_s(buf, 100);      // reads a line of input

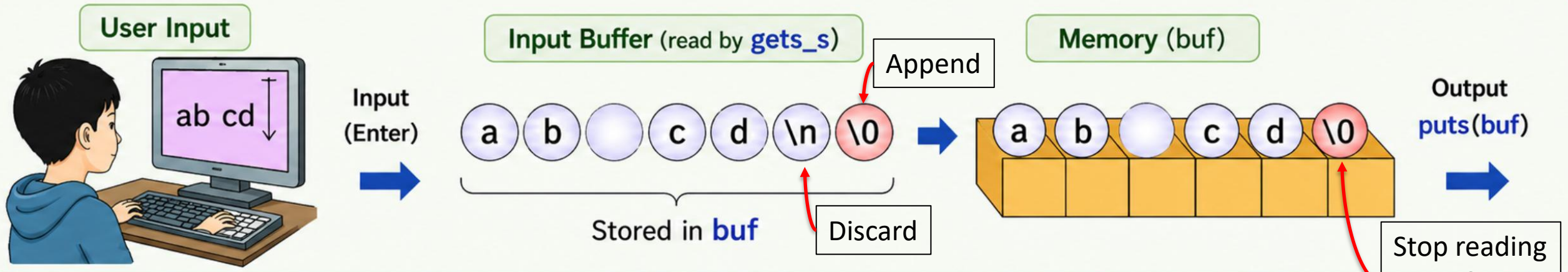
puts(buf);             // prints the line
```

gets_s(buf, size)

- Reads a line (until Enter) from the user.
- Stores it in **buf** (including '\0').

puts(buf)

- Prints the string stored in **buf**.
- Automatically adds a newline at the end.



Summary

- **gets_s(buf, size):** Reads a line from input and stores it in **buf** (appends '\0').
- **puts(buf):** Prints the string in **buf** and adds a newline.

String Input/Output Library: printf(), scanf()

```
printf("format string", arg1, arg2, ...);

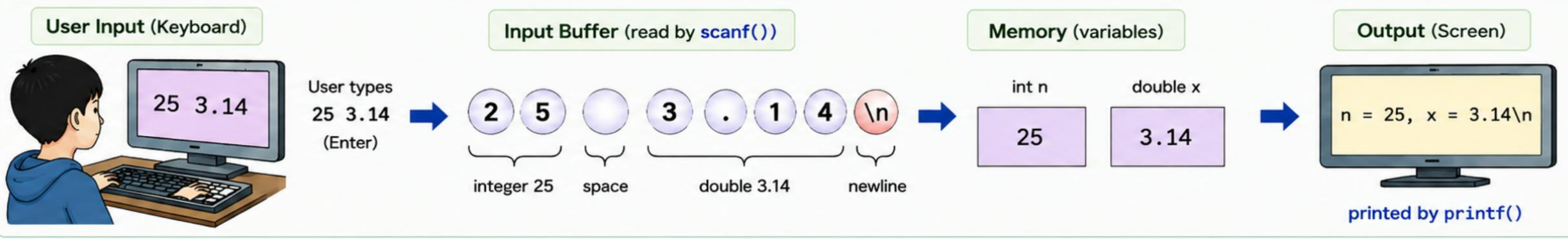
scanf("format string", &var1, &var2, ...);
```

printf()

- Prints formatted output to the screen.
- Returns the number of characters printed.

scanf()

- Reads formatted input from the keyboard.
- Stores values in the provided variables.
- Returns the number of items successfully read.



printf() Example

```
int n = 25;
double x = 3.14;
printf("n = %d, x = %.2f\n", n, x);
```

Common Format Specifiers

%d	→	integer
%f	→	floating point
%c	→	character
%s	→	string
%lf	→	double

scanf() Example

```
int n;
double x;
scanf("%d %lf", &n, &x);
```

Common Format Specifiers

%d	→	integer
%f	→	float
%lf	→	double
%c	→	character
%s	→	string

Summary

- `printf()` : Prints formatted output to the screen using a format string.
- `scanf()` : Reads formatted input from the keyboard and stores it in variables (use & before variable names).

Tip

In `scanf()`, spaces (or \t, \n) in the format string skip any amount of whitespace (space, tab, newline).

Caution: scanf("%s", ...) does not save the FULL address

```c
#include < stdio.h >

int main( void )
{
    char name[100];
    char address[100];

    printf ( "Enter your name: " );
    scanf ( "%s" , name);
    printf ( "Enter your current address: " );
    scanf ( "%s" , address);

    printf ( "Hello, Mr. %s who lives in %s.\n" , name, address);
    return 0;
}
// Caution: scanf("%s", ...) does not save the FULL address
// if it contains spaces - it stops at the first space!
```

String Input: gets_s() vs. fgets() vs. scanf()

VSC

How gets_s(), fgets(), and scanf() Handle the Input "i am a boy"

Keyboard Input

"i am a boy"
(then press Enter)



Input Buffer (stdin)

i		a	m		a		b	o	y	\n

- The input buffer contains the characters you typed plus the newline character \n from pressing Enter.
- There is no \0 (null character) in the input buffer.

Function	Stored in Memory (char str[...])	Result (string)	Explanation
gets_s(str, size)	i a m a b o y \0	"i am a boy"	• Reads a line of text. • Discards the newline (\n). • Appends a null character (\0).
fgets(str, size, stdin)	i a m a b o y \n \0	"i am a boy\n"	• Reads a line of text (up to size – 1). • Keeps the newline (\n) if it fits. • Appends a null character (\0).
scanf("%s", str)	i \0	"i"	• Reads a word (stops at the first whitespace, such as a space, tab, or newline). • Stores "i" and appends a null character (\0).

Key Takeaways

- **gets_s()**: Read a line, remove \n, add \0 → "i am a boy"
- **fgets()**: Read a line, keep \n, add \0 → "i am a boy\n"
- **scanf("%s")**: Read one word (up to whitespace), add \0 → "i"

Character Processing Library: <ctype.h>

- Check or convert characters.

Function	explanation
isalpha (c)	Is c an English letter ? (az, AZ)
isupper (c)	Is c uppercase ? (AZ)
islower(c)	c lowercase ? (az)
isdigit (c)	Is c a number ? (0-9)
isalnum(c)	c a letter or a number ? (az, AZ, 0-9)
isxdigit(c)	Is c a hexadecimal number ? (0-9, AF, af)
isspace(c)	Is c a space character ? (' ', '\n', '\t', '\v', '\r')
ispunct(c)	Is c a punctuation character ?
isprint(c)	C a printable character ?
iscntrl(c)	Is c a control character ?
isascii(c)	c an ascii code ?

Function	explanation
toupper(c)	c to uppercase .
tolower(c)	c to lowercase .
toascii(c)	c to ASCII code .

Example: Convert Lowercase to Uppercase

```c
#include < stdio.h >
#include < ctype.h >

int main( void )
{
    int c;
    while ((c = getchar ()) != EOF)
    {
        if ( islower (c) )
            c = toupper (c);
        putchar (c);
    }
    return 0;
}
```

How to read & write string

String input/output library

String Processing Library: <string.h>

- **String** functions are declared in **string.h**
  - you must include **string.h** to use them.

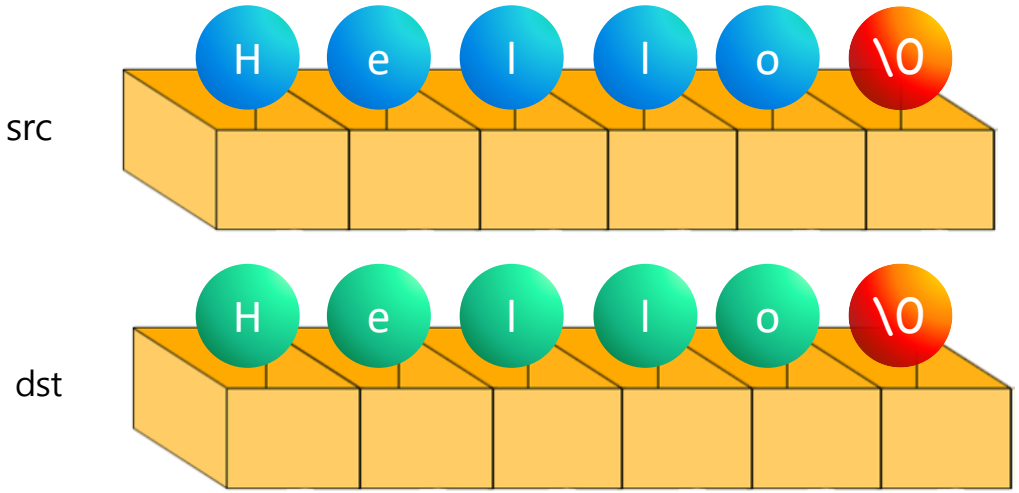
Function	explanation
strlen(s)	Finds the length of string s
strcpy(s1, s2)	Copy s2 to s1
strcat(s1, s2)	Paste s2 to the end of s1
strcmp(s1, s2)	Compare s1 and s2
strncpy(s1, s2, n)	Copy at most n characters from s2 to s1
strncat(s1, s2, n)	Paste at most n characters from s2 to the end of s1
strncmp(s1, s2, n)	Compares s1 and s2 up to n characters
strchr(s, c)	Finds the character c in string s
strstr(s1, s2)	Find string s2 in string s1

String Processing Library: String Copy

Syntax strcpy()

yes

```
char dst[6];
char src[6]="Hello";
strcpy(dst, src);                    // Copy src to dst.
```

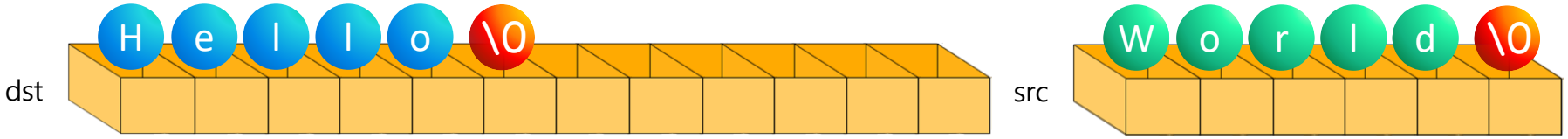


String Processing Library: String Concatenation

Syntax strcat()

yes

```
char dst[12]= "Hello";
char src[6] = "World";
strcat(dst, src);              // dst becomes "HelloWorld".
```



strcpy() and strcat()

```
#include < string.h >
#include < stdio.h >

int main( void )
{
    char string[80];
    strcpy ( string, "Hello world from " );
    strcat ( string, " strcpy " );
    strcat ( string, "and " );
    strcat ( string, " strcat!" );
    printf ( "string = %s\n" , string );
    return 0;
}
```

String Processing Library: String Comparison (strcmp())

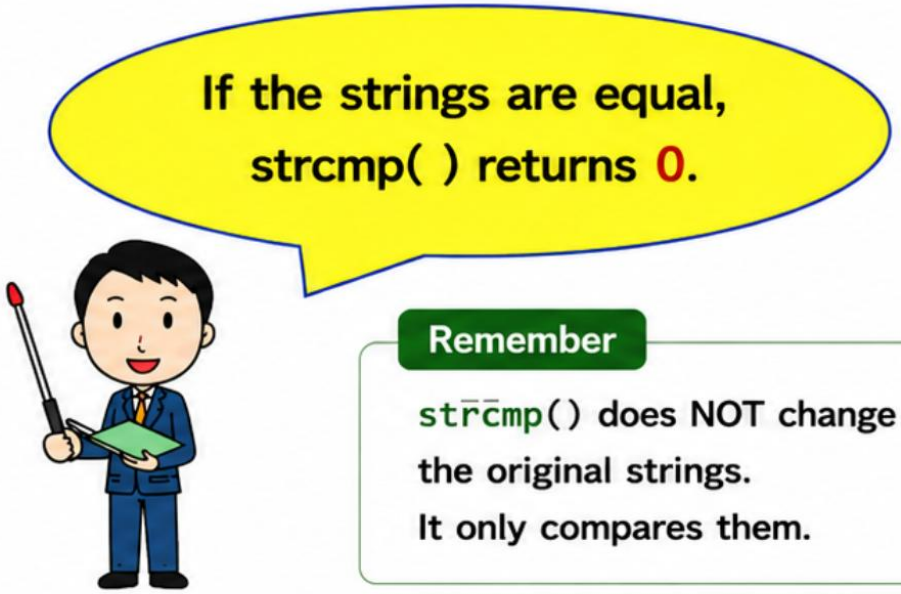
Syntax `strcmp( )`

```
int result = strcmp("dog", "dog");
```

strcmp() compares strings s1 and s2 lexicographically.

- Comparison starts at the first character.
- If all compared characters are equal and both strings end, the strings are equal.
- If a difference is found, the comparison stops and the result is returned.

Return Value	Relationship between s1 and s2	Meaning
< 0	s1 comes before s2. (s1 is lexicographically smaller than s2)	The first differing character in s1 has a smaller ASCII value than in s2.
0	s1 == s2 (strings are equal)	All characters are the same and both strings end.
> 0	s1 comes after s2. (s1 is lexicographically larger than s2)	The first differing character in s1 has a larger ASCII value than in s2.



Remember

strcmp() does NOT change the original strings.
It only compares them.

Notes / Example

- Strings are compared based on ASCII values of characters.
- strcmp() is similar but compares up to n characters.

```
strcmp("apple",  "app")     > 0    // 'l' (108)> '\0' (0)
strcmp("cat",    "dog")     < 0    // 'c' (99) <  'd' (100)
strcmp("same",   "same")    == 0   // equal
```

How comparison works (example)

s1:	d	o	g	\0
s2:	d	o	g	\0

All characters match and both end → result is 0.

strcmp(): Comparing Strings

VSC

```
int result = strcmp (s1, s2);

if ( result < 0 )
    printf ( "%s is less than %s\n" , s1, s2);
else if ( result == 0 )
    printf ( "%s and %s are the same.\n" , s1, s2);
else
    printf ( "%s is greater than %s\n" , s1, s2);
// strcmp() returns 0 when the strings are equal.
```

String Processing Library: Search a Character (strchr())

```
char *p = strchr("dog", 'g');
```

Return the address of character 'g'

What it does

- Searches for the first occurrence of character **ch** in the string **str**.
- Returns a pointer to that character in the string.
- If not found, returns **NULL**.

Return Value	Meaning	Explanation
Pointer to character	Address of the first occurrence of **ch** in **str**.	A pointer to the character found in the string.
NULL	**ch** is not found in **str**.	No such character exists in the string.

Example

```
char *p = strchr("dog", 'g');
if (p != NULL)
   printf("Found at index %ld\n", p - "dog");
else
   printf("Not found\n");
```

String index

0	1	2
d	o	g

p points here

More Examples

`strchr("banana", 'a')`

0	1	2	3	4	5
b	a	n	a	n	a

Returns address of index 1

`strchr("banana", 'n')`

0	1	2	3	4	5
b	a	n	a	n	a

Returns address of index 2

`strchr("banana", 'z')`

0	1	2	3	4	5
b	a	n	a	n	a

Returns NULL

Tips

- **strchr()** is case-sensitive.
- Include **<string,h>** to use **strchr()**.
- The returned pointer points inside the original string.

String Processing Library: Search a string (strstr())

Syntax — strstr()

```
char *p = strstr("dog and cat", "cat");
```

string s — substring sub

p points to the first occurrence of "cat" in s

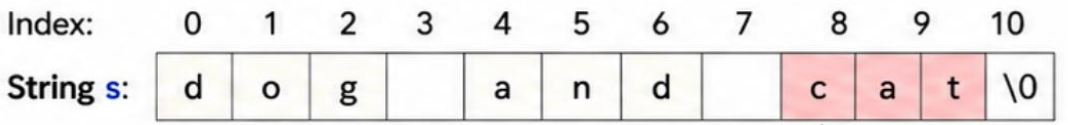
What it does

- Searches for the first occurrence of substring **sub** in string **s**.
- If found, returns the address where **sub** begins.
- If not found, returns **NULL**.

Return Value	Meaning
Pointer to char	Address of the first character of the first occurrence of sub in s.
NULL	sub is not found in s.

Example

```
char *p = strstr("dog and cat", "cat");
```



p points here
(address of index 8)

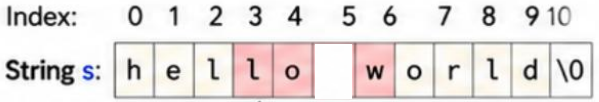
More Examples

`strstr("hello world", "world")`



Returns address of index 6

`strstr("hello world", "lo w")`



Returns address of index 3

`strstr("hello world", "abc")`



Returns NULL

Tips

- strstr() is case-sensitive.
- Include <string.h> to use strstr().
- The returned pointer points inside the original string s.

strchr(): Text Search

VSC

```c
#include < string.h >
#include < stdio.h >

int main( void )
{
    char s[] = "language";
    char c = 'g';
    char *p;
    int loc;

    p = strchr (s, c);
    if ( p == NULL )
        printf ( "%c not found\n" , c );
    else {
        loc = ( int )(p - s);
        printf ( "First %c in %s found at %d\n" , c, s, loc);
    }
    return 0;
}
```

To check whether a string contains a specific character, use strchr(). To search for a whole substring, use strstr().

String Processing Library: String Tokenization (strtok())

Syntax

```
char *p = strtok("Hello World!", " ");
```

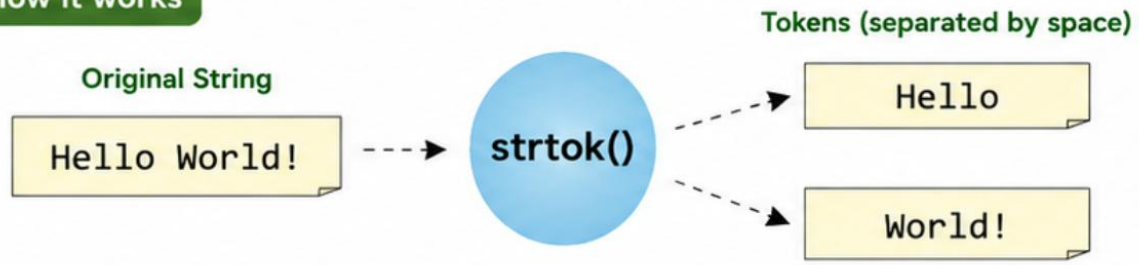
function — string to be tokenized — separator (delimiters)

Splits a string into words using spaces.

What it does

- Splits a string into tokens (words) using the specified separator characters.
- On the first call, pass the string to be tokenized.
- On subsequent calls, pass **NULL** as the first argument to get the next tokens.
- Returns a pointer to the next token found.
- When no more tokens remain, returns **NULL**.

How it works



Example

```
char *p = strtok("Hello World!", " ");

while (p != NULL) {
   printf("%s\n", p);      // print token
   p = strtok(NULL, " ");  // get next token
}
```

Output

```
Hello
World!
```

Examples with different separators

String (s)	Separator	Tokens returned (in order)
"one,two,three"	","	one → two → three
"a;b;c"	";"	a → b → c
"red|green|blue"	"|"	red → green → blue

Tips

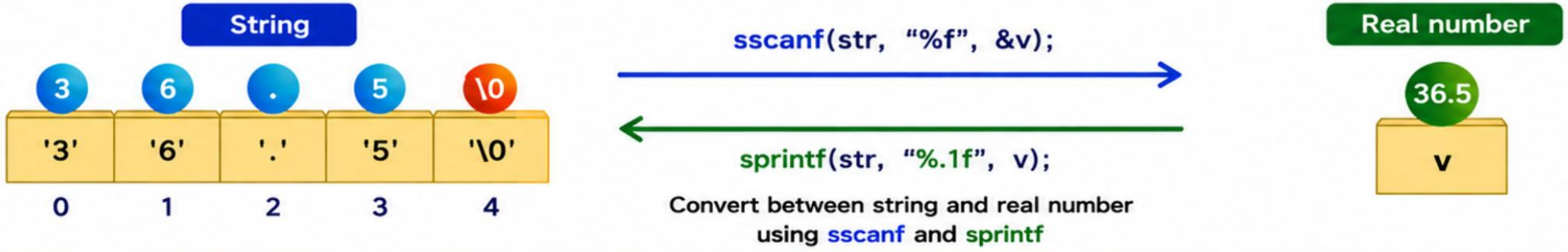
- Include <string.h> to use strtok().
- Consecutive separators are treated as one.
- The original string is modified: separators are replaced with '\0'.
- Not thread-safe (uses internal static data).

strtok(): String Seperation

VSC

```
char[] s = “Man is immortal, because he has a soul” ;
t1 = strtok (s, “ ” );        // First token :      Man            “ “ is separator.
t2 = strtok (NULL, “ ” ); // Second token : is
t3 = strtok (NULL, “ ” ); // Third token :     immortal,
t4 = strtok (NULL, “ ” ); // Fourth token :   because
t5 = strtok (NULL, “ ” ); // Fifth token :       he
t6 = strtok (NULL, “ ” ); // Sixth token :      has
t7 = strtok (NULL, “ ” ); // Seventh token : a
t8 = strtok (NULL, “ ” ); // Eighth token :    soul
t9 = strtok (NULL, “ ” ); // Ninth token :      NULL
```

String Conversion: String to Number, and vice versa



sscanf() : Convert a string to a number

- Reads the number in str according to the format and stores it in the variable pointed by &v. (sscanf is string input)

str	→	Original string containing the number
"%f"	→	Format specifier to read float (real number)
&v	→	Address of variable v to store the read value

sprintf() : Convert a number to a string

- Creates a string in str according to the format using the value of v. (sprintf is string output)

str	→	Character array to store the (numeric) string
"%.1f"	→	Format specifier for float (e.g., %.1f, %f) (specific output format)
v	→	Real number value to be converted into a string

Functions to convert a string to a number (in <stdio.h>)

Function	Explanation
int atoi(const char *str);	Convert string to int type.
double atof(const char *str);	Convert string to double type.

Example : Using atoi and atof

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
   char s1[100] = "123";
   char s2[100] = "45.67";

   int i = atoi(s1);      // convert string to int
   double d = atof(s2);   // convert string to double

   printf("i = %d\n", i);
   printf("d = %.2f\n", d);
   return 0;
}
```

Output

```
i = 123
d = 45.67
```

Common Mistakes

- A C string is just a char array ending in the **NULL terminator** '\0'
  - forgetting it causes garbage output.
- **strcpy**/**strcat** never check destination size
  - writing past the buffer is a classic **buffer overflow** bug.
- Comparing strings with **==** compares addresses, not contents
  - always use **strcmp()** instead.
- A string literal like "Hello" cannot be modified
  - use **char arr[] = "Hello"** to get a modifiable copy.

Q & A

Lab & Quiz: right after this - see you in the practice session!